Problem 1

It is common to think of $\pi^2 = 9.87$ as approximately ten. What are the absolute and relative errors in this approximation?

Solution. The exact value is $\pi^2 = 9.8696044\ldots$

The absolute error is given by:

$$|10 - \pi^2| = |10 - 9.8696044\ldots| = 0.1304$$

The relative error is given by:

$$\frac{|10 - \pi^2|}{|\pi^2|} = \frac{0.1304}{9.8696} = 0.01321 \approx 1.32\%$$

This approximation is accurate to about 2 significant figures.

Problem 2

If x and y have type double, and (fabs(x - y) >= 10) evaluates to TRUE, does that mean that y is not a good approximation to x in the sense of relative error?

Solution. No, this does not mean the approximation is poor in terms of relative error. The condition fabs(x - y) >= 10 only guarantees that the absolute error satisfies $|x - y| \geq 10$. This does not necessarily imply a poor relative error.

To see why, consider a counterexample where $x = 10^{15}$ and $y = 10^{15} - 10$. Then:

$$|x - y| = 10 \geq 10 \qquad \text{but} \qquad \frac{|x - y|}{|x|} = \frac{10}{10^{15}} = 10^{-14}$$

A relative error of 10^{-14} indicates that y agrees with x to an excellent level of approximation.

Problem 3

Show that $f_{jk} = \sin(x_0 + (j - k)\pi/3)$ satisfies the recurrence relation

$$f_{j,k+1} = f_{j,k} - f_{j+1,k}. \tag{2.15}$$

We view this as a formula that computes the f values on level $k+1$ from the f values on level k. Let $\hat{f}_{jk}$ for $k \geq 0$ be the floating point numbers that come from implementing $f_{j0} = \sin(x_0 + j\pi/3)$ and (2.15) (for $k > 0$) in double precision floating point. If

$|\hat{f}_{jk} - f_{jk}| \le \varepsilon$ for all j, show that $|\hat{f}_{j,k+1} - f_{j,k+1}| \le 2\varepsilon$ for all j. Thus, if the level k values are very accurate, then the level $k+1$ values still are pretty good.

Write a program (C++ or Python) that computes $e_k = \hat{f}_{0k} - f_{0k}$ for $1 \le k \le 60$ and $x_0 = 1$. Note that $\hat{f}_{0n}$, a single number on level n, depends on $\hat{f}_{0,n-1}$ and $\hat{f}_{1,n-1}$, two numbers on level $n-1$, and so on down to n numbers on level 0. Print the e_k and see whether they grow monotonically. Plot the e_k on a linear scale and see that the numbers seem to go bad suddenly at around $k = 50$. Plot the e_k on a log scale. For comparison, include a straight line that would represent the error if it were exactly to double each time.

Solution.

(a) Let $\theta = x_0 + (j-k)\pi/3$, so $f_{j,k} = \sin\theta$. Then:

$$f_{j+1,k} = \sin(\theta + \pi/3), \qquad f_{j,k+1} = \sin(\theta - \pi/3)$$

LHS:

$$\begin{aligned} f_{j,k} - f_{j+1,k} &= \sin\theta - \sin(\theta + \pi/3) \\ &= \sin\theta - \big[\sin\theta\cos(\pi/3) + \cos\theta\sin(\pi/3)\big] \\ &= \sin\theta - \tfrac{1}{2}\sin\theta - \tfrac{\sqrt{3}}{2}\cos\theta \\ &= \tfrac{1}{2}\sin\theta - \tfrac{\sqrt{3}}{2}\cos\theta \end{aligned}$$

RHS:

$$f_{j,k+1} = \sin(\theta - \pi/3) = \sin\theta\cos(\pi/3) - \cos\theta\sin(\pi/3) = \tfrac{1}{2}\sin\theta - \tfrac{\sqrt{3}}{2}\cos\theta$$

LHS = RHS. □

(b) The computed recurrence gives $\hat{f}_{j,k+1} = \hat{f}_{j,k} - \hat{f}_{j+1,k}$. The exact relation is $f_{j,k+1} = f_{j,k} - f_{j+1,k}$. Subtracting:

$$\hat{f}_{j,k+1} - f_{j,k+1} = \big(\hat{f}_{j,k} - f_{j,k}\big) - \big(\hat{f}_{j+1,k} - f_{j+1,k}\big)$$

By the triangle inequality:

$$|\hat{f}_{j,k+1} - f_{j,k+1}| \le |\hat{f}_{j,k} - f_{j,k}| + |\hat{f}_{j+1,k} - f_{j+1,k}| \le \varepsilon + \varepsilon = 2\varepsilon \quad \square$$

Starting from level-0 errors of order $\varepsilon_{\text{mach}}$, after k levels the error grows to $2^k \cdot \varepsilon_{\text{mach}}$. Full accuracy is lost when $2^k \cdot \varepsilon_{\text{mach}} \approx 1$, i.e., $k \approx 53$ for double precision.

(c) To compute $\hat{f}_{0,k}$, initialize $k+1$ values at level 0: $f_{j,0} = \sin(x_0 + j\pi/3)$ for $j = 0, \ldots, k$, then apply $f_{j,\ell+1} = f_{j,\ell} - f_{j+1,\ell}$ for $\ell = 0, \ldots, k-1$.

```python
import numpy as np
import matplotlib.pyplot as plt

def compute_errors(x0=1.0, max_k=60):
    errors = []
    for k in range(1, max_k + 1):
        f_hat = np.array([np.sin(x0 + j*np.pi/3.0)
                          for j in range(k + 1)])
        for level in range(k):
            f_hat = f_hat[:-1] - f_hat[1:]
        f_exact = np.sin(x0 - k*np.pi/3.0)
        errors.append(f_hat[0] - f_exact)
    return errors

errors = compute_errors()
ks = np.arange(1, 61)
eps = np.finfo(np.float64).eps

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(ks, errors, 'b.-', markersize=3)
axes[0].set_xlabel('k'); axes[0].set_ylabel('$e_k$')
axes[0].set_title('Linear Scale')

axes[1].semilogy(ks, np.abs(errors), 'b.-', markersize=3, label='$|e_k|$')
axes[1].semilogy(ks, 2.0**ks * eps, 'r--', label='$2^k \\varepsilon_{mach}$')
axes[1].set_xlabel('k'); axes[1].set_ylabel('$|e_k|$')
axes[1].set_title('Log Scale'); axes[1].legend()
plt.tight_layout(); plt.savefig('problem3_plots.png', dpi=150)
```

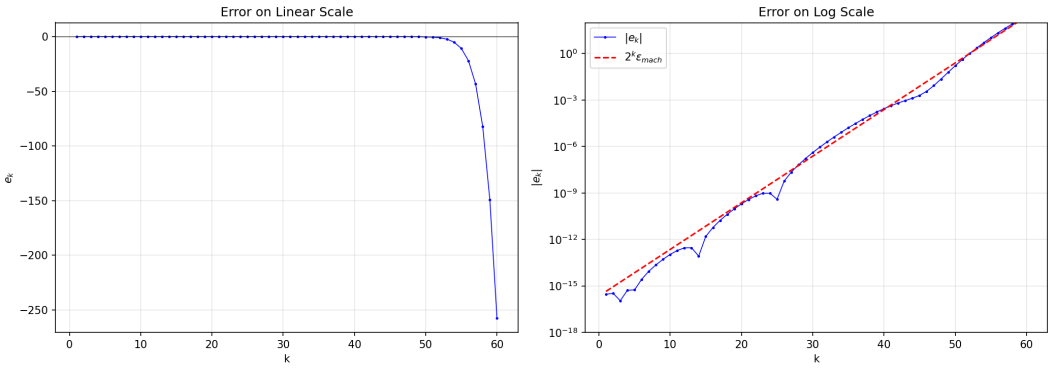

Figure 1: Left: e_k on linear scale. Right: $|e_k|$ on log scale with comparison line $2^k\varepsilon_{\text{mach}}$.

According to the plots, I have the observations:

- The errors $|e_k|$ are not monotonically increasing because oscillations occur when rounding errors change sign.
- On a linear scale, errors appear zero until approximately $k \approx 50$, at which point

they explode suddenly.

- On a log scale, $|e_k|$ grows roughly as $2^k \cdot \varepsilon_{\text{mach}}$, which confirms our theoretical bound.

Problem 4

What are the possible values of k after the for loop is finished?

```
float x = 100.0*rand() + 2;
int n = 20, k = 0;
float dy = x/n;
for (float y = 0; y < x; y += dy)
    k++;   /* Body does not change x, y, or dy */
```

Solution. In exact arithmetic, dy $= x/20$ and the loop executes exactly 20 times. In floating point, dy $= \text{fl}(x/20) = (x/20)(1+\delta)$ where $|\delta| \lesssim \varepsilon_{\text{mach}} \approx 1.2 \times 10^{-7}$.

The possible values are:

$$k \in \{20,\ 21\}$$

- $k = 20$: when $\text{fl}(x/n)$ rounds up or is exact, $20 \cdot \text{dy} \geq x$ and the loop exits normally.
- $k = 21$: when $\text{fl}(x/n)$ rounds down, $20 \cdot \text{dy} < x$ and one extra iteration occurs. The 21st iteration always exits because $y_{21} \approx 1.05x \gg x$.

$k = 19$ is not possible because it would require $19 \cdot \text{dy} \geq x$, i.e., dy $\geq x/19$. Since dy $\approx x/20$, this would need a relative error of about $20/19 - 1 \approx 5\%$, which far exceeds single-precision rounding error ($\sim 10^{-7}$). By the same argument, $k = 22$ is not possible because $21 \cdot \text{dy} < x$ would require dy to be about 5% smaller than $x/20$.

Problem 5

We wish to evaluate the function $f(x)$ for x values around 10^{-3}. We expect f to be about 10^5 and f' to be about 10^{10}. Is the problem too ill-conditioned for single precision? For double precision?

Solution. The condition number is given by:

$$\kappa = \left| \frac{x f'(x)}{f(x)} \right| \approx \frac{10^{-3} \cdot 10^{10}}{10^5} = \frac{10^7}{10^5} = 100$$

A relative perturbation δ in x produces a relative change of approximately $\kappa \cdot \delta$ in $f(x)$, which results in a loss of $\log_{10}(\kappa) = 2$ decimal digits.

Single precision has $\varepsilon_{\text{mach}} \approx 6 \times 10^{-8}$, which corresponds to approximately 7 decimal digits:

$$\text{Relative error} \approx 100 \times 6 \times 10^{-8} = 6 \times 10^{-6}$$

This means we retain approximately 5 digits, so the problem is not too ill-conditioned for single precision.

Double precision has $\varepsilon_{\text{mach}} \approx 1.1 \times 10^{-16}$, which corresponds to approximately 16 decimal digits:

$$\text{Relative error} \approx 100 \times 1.1 \times 10^{-16} = 1.1 \times 10^{-14}$$

This means we retain approximately 14 digits, which gives excellent accuracy for double precision.

Problem 6

Use the fact that the floating point sum $x + y$ has relative error $\varepsilon < \varepsilon_{\text{mach}}$ to show that the absolute error in the sum S computed below is no worse than $(n-1)\varepsilon_{\text{mach}} \sum_{k=0}^{n-1} |x_i|$:

```
double compute_sum(double x[], int n) {
    double S = 0;
    for (int i = 0; i < n; ++i)
        S += x[i];
    return S;
}
```

Solution. We prove by mathematical induction that the absolute error after summing n numbers satisfies

$$|\hat{S}_{n-1} - S_{n-1}| \leq (n-1)\varepsilon_{\text{mach}} \sum_{i=0}^{n-1} |x_i|.$$

(For $n = 1$).There is no addition, so $\hat{S}_0 = x_0 = S_0$ and the error is zero. The bound $0 \cdot \varepsilon_{\text{mach}} \cdot |x_0| = 0$ holds trivially.

(For $n = 2$). One floating point addition gives $\hat{S}_1 = (x_0 + x_1)(1 + \delta_1)$ with $|\delta_1| \leq \varepsilon_{\text{mach}}$. Thus:

$$|\hat{S}_1 - S_1| = |x_0 + x_1||\delta_1| \leq \varepsilon_{\text{mach}}(|x_0| + |x_1|) = 1 \cdot \varepsilon_{\text{mach}} \sum_{i=0}^{1} |x_i|.$$

(Inductive step) Assume the bound holds for $n - 1$ terms:

$$|\hat{S}_{n-2} - S_{n-2}| \leq (n-2)\varepsilon_{\text{mach}} \sum_{i=0}^{n-2} |x_i|.$$

Adding the n-th element produces $\hat{S}_{n-1} = (\hat{S}_{n-2} + x_{n-1})(1 + \delta_{n-1})$ with $|\delta_{n-1}| \leq \varepsilon_{\text{mach}}$. Expanding and subtracting the exact sum $S_{n-1} = S_{n-2} + x_{n-1}$:

$$\hat{S}_{n-1} - S_{n-1} = (\hat{S}_{n-2} - S_{n-2}) + (\hat{S}_{n-2} + x_{n-1})\delta_{n-1}.$$

Taking absolute values and applying the triangle inequality:

$$|\hat{S}_{n-1} - S_{n-1}| \leq |\hat{S}_{n-2} - S_{n-2}| + |\hat{S}_{n-2} + x_{n-1}||\delta_{n-1}|.$$

Since $|\hat{S}_{n-2} + x_{n-1}| \leq |S_{n-2} + x_{n-1}| + |\hat{S}_{n-2} - S_{n-2}| \leq \sum_{i=0}^{n-1} |x_i| + |\hat{S}_{n-2} - S_{n-2}|$, and since $(n-2)\varepsilon_{\text{mach}} \ll 1$ for practical n, we have $|\hat{S}_{n-2} + x_{n-1}| \leq \sum_{i=0}^{n-1} |x_i|$ to first order. Substituting the inductive hypothesis:

$$\begin{aligned}
|\hat{S}_{n-1} - S_{n-1}| &\leq (n-2)\varepsilon_{\text{mach}} \sum_{i=0}^{n-2} |x_i| + \varepsilon_{\text{mach}} \sum_{i=0}^{n-1} |x_i| \\
&\leq (n-2)\varepsilon_{\text{mach}} \sum_{i=0}^{n-1} |x_i| + \varepsilon_{\text{mach}} \sum_{i=0}^{n-1} |x_i| \\
&= (n-1)\varepsilon_{\text{mach}} \sum_{i=0}^{n-1} |x_i|. \quad \square
\end{aligned}$$

Problem 7

Starting with the declarations

```
float x, y, z, w;
const float oneThird = 1/ (float) 3;
const float oneHalf  = 1/ (float) 2;
// const means these never are reassigned
```

we do lots of arithmetic on the variables x, y, z, w. In each case below, determine whether the two arithmetic expressions result in the same floating point number (down to the last bit) as long as no NaN or inf values or denormalized numbers are produced.

(a) (x * y) + (z - w) and (z - w) + (y * x)

(b) (x + y) + z and x + (y + z)

(c) x * oneHalf + y * oneHalf and (x + y) * oneHalf

(d) x * oneThird + y * oneThird and (x + y) * oneThird

Solution.

(a) Same. IEEE 754 multiplication is commutative, which means $\text{fl}(x \cdot y) = \text{fl}(y \cdot x)$ since the mathematical product $x \cdot y = y \cdot x$ is unique and rounding a unique value gives

a unique result. If we let $P = \text{fl}(xy)$ and $Q = \text{fl}(z - w)$, then both expressions compute $\text{fl}(P + Q) = \text{fl}(Q + P)$ since addition is also commutative.

(b) Maybe Different. Floating point addition is not associative.

Counterexample in single precision: Let $x = 1.0$, $y = 2^{-24}$, and $z = -1.0$. Then:

$$\begin{aligned} (x + y) + z: &\quad \text{fl}(1.0 + 2^{-24}) = 1.0 \text{ due to absorption}, \quad \text{fl}(1.0 - 1.0) = 0.0 \\ x + (y + z): &\quad \text{fl}(2^{-24} - 1.0) \approx -1.0 + 2^{-24}, \quad \text{fl}(1.0 + (-1.0 + 2^{-24})) \approx 2^{-24} \end{aligned}$$

(c) Maybe Different. oneHalf $= 0.5 = 2^{-1}$ is exact, and multiplication by 0.5 is exact since it only decrements the exponent.

$$\text{LHS} = \text{fl}\big(\underbrace{x/2}_{\text{exact}} + \underbrace{y/2}_{\text{exact}}\big) = \text{fl}\left(\frac{x+y}{2}\right)$$

$$\text{RHS} = \underbrace{\text{fl}(x+y)}_{\text{rounded}} \cdot 0.5 = \frac{\text{fl}(x+y)}{2}$$

These expressions differ when $\text{fl}\big(\frac{x+y}{2}\big) \neq \frac{\text{fl}(x+y)}{2}$, which can happen because first rounding the sum and then halving is not the same as first halving each term and then rounding the sum.

(d) Maybe Different. oneThird $= \text{fl}(1/3) \neq 1/3$ since $1/3$ has a non-terminating binary representation. Because multiplication by oneThird is not exact, the distributive law fails:

$$\text{fl}(x \cdot c) + \text{fl}(y \cdot c) \neq \text{fl}\big((x + y) \cdot c\big)$$

This inequality holds in general, particularly when the constant c cannot be represented exactly in floating point.

Problem 8

The Fibonacci numbers, f_k, are defined by $f_0 = 1$, $f_1 = 1$, and

$$f_{k+1} = f_k + f_{k-1} \tag{2.16}$$

for any integer $k > 1$. A small perturbation of them, the pib numbers ("p" instead of "f" to indicate a perturbation), p_k, are defined by $p_0 = 1$, $p_1 = 1$, and

$$p_{k+1} = c \cdot p_k + p_{k-1}$$

for any integer $k > 1$, where $c = 1 + \sqrt{3}/100$.

(a) Plot the f_n and p_n together on a log scale plot. On the plot, mark $1/\varepsilon_{\text{mach}}$ for single and double precision arithmetic. This can be useful in answering the questions below.

(b) Rewrite (2.16) to express f_{k-1} in terms of f_k and f_{k+1}. Use the computed f_n and f_{n-1} to recompute f_k for $k = n-2, n-3, \ldots, 0$. Make a plot of the difference between the original $f_0 = 1$ and the recomputed f_0 as a function of n. What n values result in no accuracy for the recomputed f_0? How do the results in single and double precision differ?

(c) Repeat (b) for the pib numbers. Comment on the striking difference in the way precision is lost in these two cases. Which is more typical?

Extra credit: predict the order of magnitude of the error in recomputing p_0 using what you may know about recurrence relations and what you should know about computer arithmetic.

Solution.

(a) Both sequences grow exponentially. The Fibonacci numbers grow as $f_n \sim \varphi^n/\sqrt{5}$ where $\varphi = (1+\sqrt{5})/2 \approx 1.618$, and the pib numbers grow as $p_n \sim r_1^n$ where $r_1 \approx 1.635$. We compute both sequences and plot them on a log scale, with horizontal lines showing $1/\varepsilon_{\text{mach}}$ for each precision level.

```
import numpy as np
import matplotlib.pyplot as plt

c_val = 1.0 + np.sqrt(3.0) / 100.0
eps_s = np.finfo(np.float32).eps
eps_d = np.finfo(np.float64).eps
n_max = 100

f_d = np.zeros(n_max+1, dtype=np.float64); f_d[0]=f_d[1]=1.0
p_d = np.zeros(n_max+1, dtype=np.float64); p_d[0]=p_d[1]=1.0
for k in range(2, n_max+1):
    f_d[k] = f_d[k-1] + f_d[k-2]
    p_d[k] = c_val*p_d[k-1] + p_d[k-2]

ns = np.arange(n_max+1)
plt.semilogy(ns, f_d, 'b-', label='Fibonacci')
plt.semilogy(ns, p_d, 'r-', label='Pib')
plt.axhline(1/eps_s, color='g', ls='--', label=f'$1/\\varepsilon_{{mach}}$
    ↪ single')
plt.axhline(1/eps_d, color='purple', ls='--', label=f'$1/\\varepsilon_{{
    ↪ mach}}$ double')
plt.legend(); plt.xlabel('n'); plt.ylabel('Value')
plt.savefig('problem8a_plot.png', dpi=150)
```

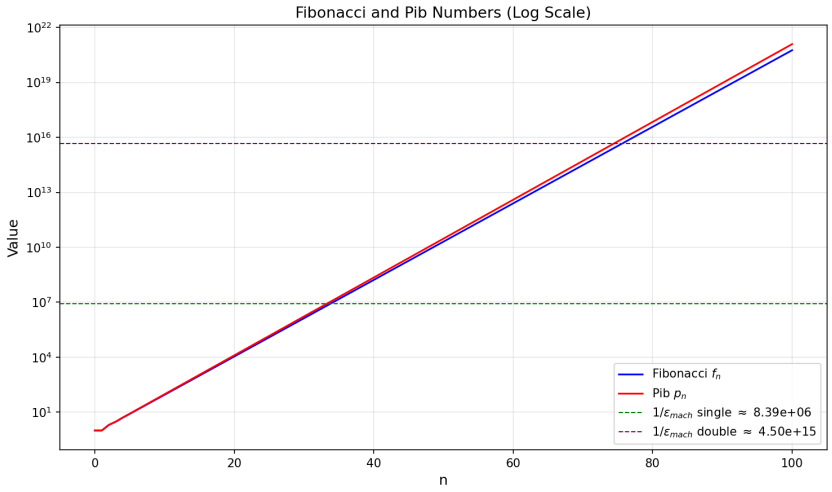

Figure 2: Fibonacci and Pib on log scale. Horizontal lines: $1/\varepsilon_{\text{mach}}$ for single ($\approx 8.4\times10^6$) and double ($\approx 4.5 \times 10^{15}$).

(b) The backward recurrence can be written as $f_{k-1} = f_{k+1} - f_k$. I implement this with separate functions for single and double precision:

```
def backward_fib_double(n):
    f = np.zeros(n+1, dtype=np.float64)
    f[0] = f[1] = 1.0
    for k in range(2, n+1):
        f[k] = f[k-1] + f[k-2]
    fk1, fk = f[n], f[n-1]
    for k in range(n-2, -1, -1):
        fk1, fk = fk, fk1 - fk
    return abs(fk - 1.0)

def backward_fib_single(n):
    f = np.zeros(n+1, dtype=np.float32)
    f[0] = f[1] = np.float32(1.0)
    for k in range(2, n+1):
        f[k] = f[k-1] + f[k-2]
    fk1, fk = np.float32(f[n]), np.float32(f[n-1])
    for k in range(n-2, -1, -1):
        fk1, fk = fk, np.float32(fk1 - fk)
    return abs(float(fk) - 1.0)

ns_d = range(2, 95)
ns_s = range(2, 50)
err_fib_d = [backward_fib_double(n) for n in ns_d]
err_fib_s = [backward_fib_single(n) for n in ns_s]

plt.semilogy(list(ns_d), err_fib_d, 'b.-', ms=3, label='Double')
plt.semilogy(list(ns_s), err_fib_s, 'r.-', ms=3, label='Single')
plt.axhline(1.0, color='k', ls='--', lw=0.5, label='Complete loss')
plt.xlabel('n'); plt.ylabel('|recomputed $f_0$ - 1|')
```

```
plt.legend(); plt.savefig('problem8b_plot.png', dpi=150)
```

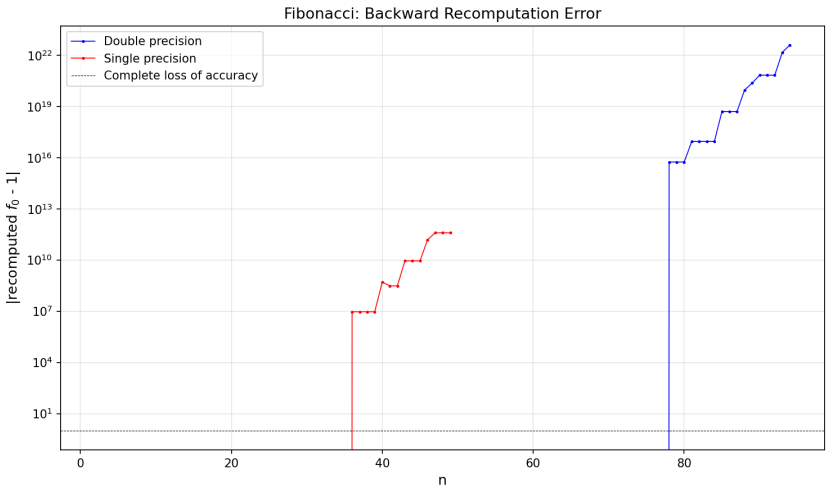

Figure 3: Fibonacci backward recomputation error. Red: single precision. Blue: double precision.

Fibonacci numbers are integers. Integer addition and subtraction in floating point is exact as long as all values are less than 2^p, where $p = 24$ for single precision and $p = 53$ for double precision. Therefore:

- When n is small enough that $f_n < 2^p$, the error is exactly zero.
- When n is large and values exceed 2^p, rounding errors appear and are amplified by the backward recurrence. Since the growing parasitic solution has growth rate φ^n, errors amplify as approximately $\varphi^{2n} \cdot \varepsilon_{\text{mach}}$.

Comparing single and double precision:

- Single precision has $2^{24} \approx 1.7 \times 10^7$, so error is exactly zero for $n \lesssim 35$ and reaches complete loss (error ≥ 1) by $n \approx 40$.
- Double precision has $2^{53} \approx 9 \times 10^{15}$, so error is exactly zero for $n \lesssim 75$ and reaches complete loss by $n \approx 80$.

The behavior is qualitatively the same: a sharp transition from zero error to total loss within just a few steps, due to the exponential amplification factor φ^{2n}. The transition simply shifts later in double precision because exact integer arithmetic extends to a much wider range.

(c) The backward pib recurrence is $p_{k-1} = p_{k+1} - c \cdot p_k$. We repeat the same experiment in both precisions:

```
c32 = np.float32(c_val)

def backward_pib_double(n):
```

```
    p = np.zeros(n+1, dtype=np.float64)
    p[0] = p[1] = 1.0
    for k in range(2, n+1):
        p[k] = c_val * p[k-1] + p[k-2]
    pk1, pk = p[n], p[n-1]
    for k in range(n-2, -1, -1):
        pk1, pk = pk, pk1 - c_val * pk
    return abs(pk - 1.0)

def backward_pib_single(n):
    p = np.zeros(n+1, dtype=np.float32)
    p[0] = p[1] = np.float32(1.0)
    for k in range(2, n+1):
        p[k] = c32 * p[k-1] + p[k-2]
    pk1, pk = np.float32(p[n]), np.float32(p[n-1])
    for k in range(n-2, -1, -1):
        pk1, pk = pk, np.float32(pk1 - c32 * pk)
    return abs(float(pk) - 1.0)

err_pib_d = [backward_pib_double(n) for n in ns_d]
err_pib_s = [backward_pib_single(n) for n in ns_s]

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].semilogy(list(ns_d), err_fib_d, 'b.-', ms=3, label='Double')
axes[0].semilogy(list(ns_s), err_fib_s, 'r.-', ms=3, label='Single')
axes[0].set_title('Fibonacci'); axes[0].legend()

axes[1].semilogy(list(ns_d), err_pib_d, 'b.-', ms=3, label='Double')
axes[1].semilogy(list(ns_s), err_pib_s, 'r.-', ms=3, label='Single')
axes[1].set_title('Pib'); axes[1].legend()
plt.savefig('problem8c_plot.png', dpi=150)
```

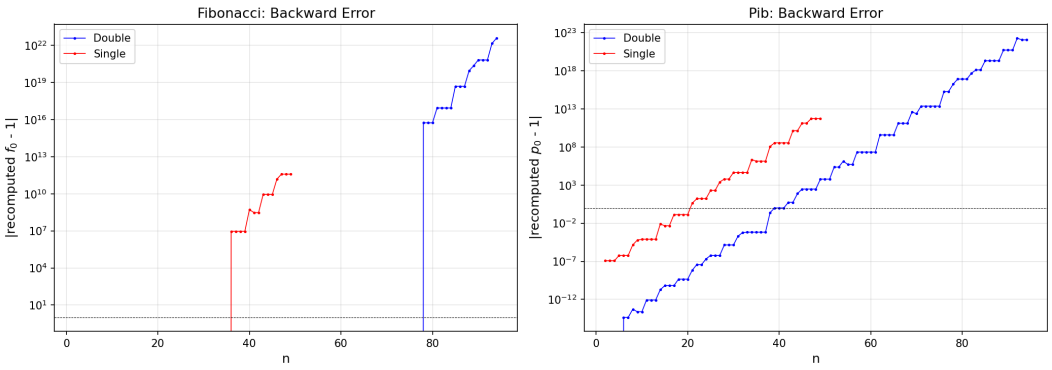

Figure 4: Left: Fibonacci backward error. Right: Pib backward error. Red: single. Blue: double.

The pib numbers show gradual, continuous error growth from $n = 2$ onward in both precisions. This occurs because $c = 1 + \sqrt{3}/100$ is irrational, which means $\text{fl}(c) \neq c$ and there is no range where exact computation is possible.

The striking difference between Fibonacci and pib:

- **Fibonacci:** Error is exactly zero until a sharp threshold is reached at approximately $n \approx 35$ for single precision and $n \approx 75$ for double precision, at which point sudden catastrophic blowup occurs. This phenomenon happens because Fibonacci numbers are integers and integer arithmetic is exact within the representable range.
- **Pib:** Error grows continuously from $n = 2$ onward. Single precision loses all accuracy around $n \approx 20$, while double precision loses accuracy around $n \approx 37$. There is no abrupt transition.

The pib case is more typical of real-world recurrences since coefficients are rarely exact integers. In both cases, switching from single to double precision only delays the inevitable loss of accuracy. The growth rate of the error remains the same, and only $\varepsilon_{\text{mach}}$ changes.

Extra credit: The characteristic equation $r^2 - cr - 1 = 0$ has roots r_1 and r_2 satisfying $r_1 r_2 = -1$, which implies $|r_1/r_2| = r_1^2$. The backward error grows as r_1^{2n}. When we combine this with the level-n roundoff error, which is approximately $\varepsilon_{\text{mach}} \cdot r_1^n$, we obtain:

$$\text{error} \sim \varepsilon_{\text{mach}} \cdot r_1^{2n} \approx 10^{-16} \cdot 1.635^{2n} = 10^{-16} \cdot 2.673^n.$$

Total loss of accuracy occurs when $2.673^n \approx 10^{16}$, which gives $n \approx 16/\log_{10}(2.673) \approx 37$. This matches our observations. For single precision, we replace 10^{-16} with 10^{-7} and find total loss at $n \approx 7/0.427 \approx 16$, which is also consistent with our results.

Problem 9

The binomial coefficients, $a_{n,k}$, are defined by

$$a_{n,k} = \binom{n}{k} = \frac{n!}{k!(n-k)!}$$

To compute the $a_{n,k}$ for a given n, start with $a_{n,0} = 1$ and then use the recurrence relation $a_{n,k+1} = \frac{n-k}{k+1}\, a_{n,k}$.

(a) For a range of n values, compute the $a_{n,k}$ this way, noting the largest $a_{n,k}$ and the accuracy with which $a_{n,n} = 1$ is computed. Do this in single and double precision. Why is roundoff not a problem here as it was in problem 8? Find n values for which $a_{n,n} \approx 1$ in double precision but not in single precision. How is this possible, given that roundoff is not a problem?

(b) Use the algorithm of part (a) to compute

$$E(k) = \frac{1}{2^n} \sum_{k=0}^{n} k\, a_{n,k} = \frac{n}{2}. \tag{2.17}$$

Write a program without any safeguards against overflow or zero divide (this time only!). Show (both in single and double precision) that the computed answer has high accuracy as long as the intermediate results are within the range of floating point numbers. As with (a), explain how the computer gets an accurate, small, answer when the intermediate numbers have such a wide range of values. Why is cancellation not a problem? Note the advantage of a wider range of values: we can compute $E(k)$ for much larger n in double precision. Print $E(k)$ as computed by (2.17) and $M_n = \max_k a_{n,k}$. For large n, one should be inf and the other NaN. Why?

(c) For fairly large n, plot $a_{n,k}/M_n$ as a function of k for a range of k chosen to illuminate the interesting "bell shaped" behavior of the $a_{n,k}$ near $k = n/2$. Combine the curves for $n = 10$, $n = 20$, and $n = 50$ in a single plot. Choose the three k ranges so that the curves are close to each other. Choose different line styles for the three curves.

Solution.

(a) The recurrence $a_{n,k+1} = r_k \cdot a_{n,k}$ with $r_k = (n-k)/(k+1)$ is first-order and multiplicative. It has only one solution, which means there is no parasitic growing mode. The ratio r_k is greater than 1 for $k < (n-1)/2$ and less than 1 for $k > (n-1)/2$. Therefore, errors introduced during the growth phase get shrunk during the decay phase.

For n greater than approximately 130, the peak value $a_{n,\lfloor n/2 \rfloor}$ exceeds 3.4×10^{38}, which is the maximum representable value in single precision and causes overflow. Double precision can handle values up to approximately 10^{308}, which allows n values up to around 1020. The limiting factor is overflow, which is a range issue, not roundoff, which is a precision issue.

```
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore', category=RuntimeWarning)

def binom_coeffs(n, dtype=np.float64):
    a = np.zeros(n+1, dtype=dtype)
    a[0] = dtype(1.0)
    for k in range(n):
        a[k+1] = a[k] * dtype(n-k) / dtype(k+1)
    return a

for n in [10, 20, 30, 50, 100, 130, 150, 200, 500, 1000]:
    ad = binom_coeffs(n, np.float64)
    as_ = binom_coeffs(n, np.float32)
    print(f"n={n:6d}  max(double)={np.max(ad):15.6e}  "
          f"a_nn(double)={ad[n]:18.10e}  a_nn(single)={as_[n]:18.10e}")
```

n	$\max_k a_{n,k}$ (double)	$a_{n,n}$ (double)	$\max_k a_{n,k}$ (single)	$a_{n,n}$ (single)
10	2.52×10^{2}	1.0000000000	2.52×10^{2}	1.0000000000
20	1.85×10^{5}	1.0000000000	1.85×10^{5}	1.0000000000
30	1.55×10^{8}	1.0000000000	1.55×10^{8}	1.0000000000
50	1.26×10^{14}	1.0000000000	1.26×10^{14}	0.9999999404
100	1.01×10^{29}	1.0000000000	1.01×10^{29}	0.9999992251
130	9.51×10^{37}	1.0000000000	inf	inf
150	9.28×10^{43}	1.0000000000	inf	inf
200	9.05×10^{58}	1.0000000000	inf	inf
500	1.17×10^{149}	1.0000000000	inf	inf
1000	2.70×10^{299}	1.0000000000	inf	inf

(b) All terms $k \cdot a_{n,k}$ are non-negative, which means the sum is monotonically increasing and there is no subtraction of large numbers. Therefore, cancellation is not a problem.

```
for n in [10, 20, 50, 100, 200, 500, 1000, 1020, 1025, 1030]:
    a = binom_coeffs(n)
    Mn = np.max(a)
    ks = np.arange(n+1, dtype=np.float64)
    two_n = np.float64(2.0) ** np.float64(n)
    Ek = np.sum(ks * a) / two_n
    exact = n / 2.0
    err = abs(Ek - exact) if np.isfinite(Ek) else float('inf')
    print(f"n={n:6d}  E(k)={Ek:20.10e}  n/2={exact:10.1f}  "
          f"|error|={err:15.6e}  Mn={Mn:20.6e}")
```

n	$E(k)$	$n/2$	|error|	M_n
10	5.000000e+00	5.0	0.000000e+00	2.52×10^{2}
20	1.000000e+01	10.0	0.000000e+00	1.85×10^{5}
50	2.500000e+01	25.0	0.000000e+00	1.26×10^{14}
100	5.000000e+01	50.0	7.105427e-15	1.01×10^{29}
200	1.000000e+02	100.0	0.000000e+00	9.05×10^{58}
500	2.500000e+02	250.0	1.421085e-13	1.17×10^{149}
1000	5.000000e+02	500.0	3.410605e-13	2.70×10^{299}
1020	inf	510.0	inf	2.81×10^{305}
1025	NaN	512.5	inf	inf
1030	NaN	515.0	inf	inf

For $n \leq 1000$, $E(k)$ matches $n/2$ to full double precision accuracy, even though intermediate values $a_{n,k}$ span hundreds of orders of magnitude. This works because all terms are non-negative, so no cancellation occurs.

For $n = 1020$, M_n is still finite but the sum $\sum k \cdot a_{n,k}$ overflows, making $E(k) = \text{inf}$. For $n = 1025$, the peak $a_{n,n/2}$ itself exceeds 10^{308} and overflows to inf, making $M_n = \text{inf}$. The sum then involves $\text{inf} - \text{inf}$ or inf/inf operations, producing $E(k) = \text{NaN}$.

Double precision handles much larger values of n than single precision because it has a wider exponent range, allowing values up to approximately 10^{308} compared to 10^{38} for single precision.

(c) The bell-shaped peak of $a_{n,k}$ is centered at $k = n/2$ with width proportional to $\sqrt{n}$. To make the three curves overlap, we select $k \in [\,n/2 - 4\sqrt{n/4},\ n/2 + 4\sqrt{n/4}\,]$ for each n and shift the horizontal axis to $k - n/2$. This gives:

- $n = 10$: $k \in [0, 10]$, i.e. all values
- $n = 20$: $k \in [1, 19]$
- $n = 50$: $k \in [11, 39]$

On the right, the fully standardized variable $z = (k - n/2)/\sqrt{n/4}$ collapses all three curves onto the Gaussian $e^{-z^2/2}$, confirming the Central Limit Theorem.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for n, style in [(10, 'b-'), (20, 'r--'), (50, 'g-.')]:
    a = binom_coeffs(n)
    Mn = np.max(a)
    ks = np.arange(n+1)
    mu = n / 2.0
    sigma = np.sqrt(n / 4.0)
    # Left: chosen k range, shifted to center
    k_lo = max(0, int(mu - 4*sigma))
    k_hi = min(n, int(mu + 4*sigma))
    mask = (ks >= k_lo) & (ks <= k_hi)
    axes[0].plot(ks[mask] - mu, a[mask]/Mn, style, linewidth=1.5, label=f'
        ↪ n = {n}')
    # Right: standardized variable
    z = (ks - mu) / sigma
    axes[1].plot(z, a/Mn, style, linewidth=1.5, label=f'n = {n}')

z_c = np.linspace(-4, 4, 200)
axes[1].plot(z_c, np.exp(-z_c**2/2), 'k:', lw=1, alpha=0.5, label='
    ↪ Gaussian')

axes[0].set_xlabel('$k - n/2$'); axes[0].set_ylabel(r'$a_{n,k}/M_n$')
axes[0].set_title('Centered at $n/2$')
axes[0].legend(); axes[0].grid(True, alpha=0.3)

axes[1].set_xlabel(r'$(k - n/2)/\sqrt{n/4}$'); axes[1].set_ylabel(r'$a_{n,
    ↪ k}/M_n$')
axes[1].set_title('Standardized (approaching Gaussian)')
axes[1].legend(); axes[1].grid(True, alpha=0.3); axes[1].set_xlim(-4, 4)

plt.tight_layout()
plt.savefig('problem9c_plot.png', dpi=150, bbox_inches='tight')
```

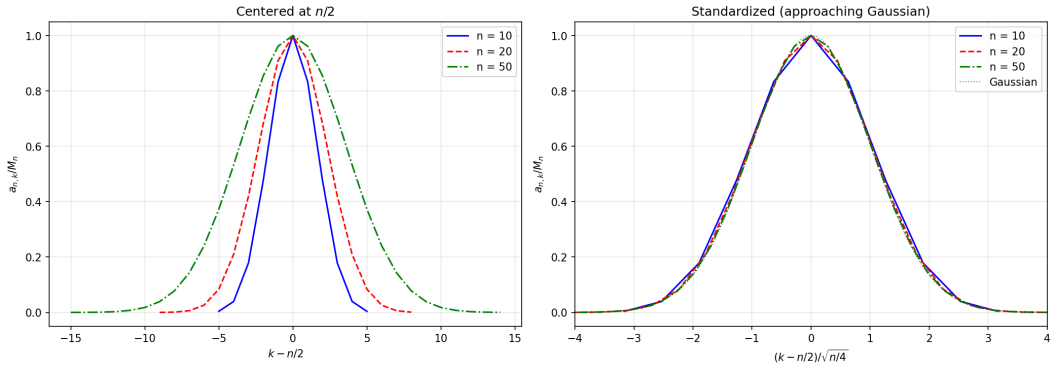

Figure 5: Left: $a_{n,k}/M_n$ vs. $k - n/2$ over chosen k ranges. Right: Standardized curves collapse onto Gaussian $e^{-z^2/2}$.